Date:-5/11/2026

# Week-5

## Grey Wolf Optimization:

### Pseudocode

BEGIN GWO

1. Initialize parameters:
   N = Number of wolves
   T = Maximum number of iterations
   lb = Lower bound
   ub = Upper bound
   D = Number of dimensions

2. Initialize wolf population randomly:

   X[i][j] = lb + rand() * (ub - lb)

   where i = 1 to N
   j = 1 to D

3. Initialize:
   Alpha_score = infinity
   Beta_score = infinity
   Delta_score = infinity

4. FOR t = 1 TO T DO

   a = 2 - (2 * t / T)

5. FOR each wolf i = 1 to N DO

   Calculate fitness: Fitness = f(X[i])

   IF Fitness < Alpha_score THEN
   Delta_score = Beta_score
   Delta_position = Beta_position

   Beta_score = Alpha_score
   Beta_position = Alpha_position

   Alpha_score = Fitness

Alpha_position = X[i]

ELSE IF Fitness < Beta_score THEN
Delta_score = Beta_score
Delta_position = Beta_position

Beta_score = Fitness
Beta_position = X[i]

ELSE IF Fitness < Delta_score THEN
Delta_score = Fitness
Delta_position = X[i]

END IF

END FOR

6. FOR each wolf i = 1 TO N DO

Generate random numbers r1 and r2

A1 = 2*a*r1 - a
C1 = 2*r2

D_alpha = |C1*Alpha_position - X[i]|

X1 = Alpha_position - A1*D_alpha

Generate random numbers r1 and r2

A2 = 2*a*r1 - a
C2 = 2*r2

D_beta = |C2*Beta_position - X[i]|

X2 = Beta_position - A2*D_beta

Generate random numbers r1 and r2

A3 = 2*a*r1 - a
C3 = 2*r2

D_delta = |C3*Delta_position - X[i]|

X3 = Delta_position - A3*D_delta

Update wolf position:

X[i] = (X1 + X2 + X3) / 3

Apply boundary limits to X[i]

END FOR

7. END FOR

8. OUTPUT:
Alpha_position = Best solution
Alpha_score = Best fitness value

END GWO

## Python Program: Size Optimization of Truss Structures Using Improved Grey Wolf Optimizer

```
import numpy as np


# 1. Truss Structure Information


E = 200e9            # Young's modulus (Pa)

rho = 7850           # Density of steel (kg/m3)


stress_limit = 250e6   # Allowable stress (Pa)

disp_limit = 0.02      # Allowable displacement (m)


# Coordinates of 3 nodes (x, y)

nodes = np.array([

   [0, 0],

   [4, 0],
```

```
    [2, 3]
])

# Connections between nodes
members = [
    [0, 1],
    [0, 2],
    [1, 2]
]

# External load (N)
F = np.zeros(6)
F[5] = -100000

# 2. Structural Analysis
def analyze_truss(areas):
    K = np.zeros((6, 6))
    lengths = []
    directions = []

    for i, (n1, n2) in enumerate(members):
        x1, y1 = nodes[n1]
        x2, y2 = nodes[n2]
        dx = x2 - x1
        dy = y2 - y1

        L = np.sqrt(dx**2 + dy**2)
        c = dx / L
        s = dy / L
```

```
    lengths.append(L)
    directions.append((c, s))

    k = (E * areas[i] / L) * np.array([
        [c*c, c*s, -c*c, -c*s],
        [c*s, s*s, -c*s, -s*s],
        [-c*c, -c*s, c*c, c*s],
        [-c*s, -s*s, c*s, s*s]
    ])

    dofs = [
        2*n1, 2*n1+1,
        2*n2, 2*n2+1
    ]

    for a in range(4):
        for b in range(4):
            K[dofs[a], dofs[b]] += k[a, b]

# Node 1 fixed in x and y
# Node 2 fixed in y
fixed = [0, 1, 3]
free = [2, 4, 5]

# Calculate nodal displacements
U = np.zeros(6)
U[free] = np.linalg.solve(
    K[np.ix_(free, free)],
```

```
        F[free]
    )

    # Calculate stress in each member
    stresses = []

    for i, (n1, n2) in enumerate(members):

        c, s = directions[i]

        u1 = U[2*n1]
        v1 = U[2*n1+1]
        u2 = U[2*n2]
        v2 = U[2*n2+1]
        strain = (c*(u2-u1) + s*(v2-v1) ) / lengths[i]

        stress = E * strain
        stresses.append(stress)

    weight = sum(
        rho * areas[i] * lengths[i]
        for i in range(len(members))
    )

    return weight, np.array(stresses), U

# 3. Fitness Function

def fitness(areas):
```

```
    weight, stresses, U = analyze_truss(areas)

    # Stress constraint violation
    stress_penalty = np.sum(
        np.maximum(
            0,
            np.abs(stresses)/stress_limit - 1
        )
    )

    # Displacement constraint violation
    max_disp = np.max(np.abs(U))
    disp_penalty = max(
        0,
        max_disp/disp_limit - 1
    )

    # Penalized objective function
    penalty = (
        1 + stress_penalty + disp_penalty
    ) ** 2

    return weight * penalty

# 4. Grey Wolf Optimization

def GWO():
```

```
population = 30
iterations = 200

dimensions = 3

lower = 0.0001
upper = 0.005

# Initialize wolf positions
wolves = np.random.uniform(
    lower, upper,
    (population, dimensions)
)

alpha = np.zeros(dimensions)
beta = np.zeros(dimensions)
delta = np.zeros(dimensions)

alpha_score = float("inf")
beta_score = float("inf")
delta_score = float("inf")

for t in range(iterations):

    # Evaluate fitness
    for i in range(population):

        score = fitness(wolves[i])
```

```
    if score < alpha_score:
        delta_score = beta_score
        delta = beta.copy()

        beta_score = alpha_score
        beta = alpha.copy()

        alpha_score = score
        alpha = wolves[i].copy()

    elif score < beta_score:
        delta_score = beta_score
        delta = beta.copy()

        beta_score = score
        beta = wolves[i].copy()

    elif score < delta_score:
        delta_score = score
        delta = wolves[i].copy()

# Decrease parameter a
a = 2 - t * (2 / iterations)

# Update wolf positions
for i in range(population):

    for j in range(dimensions):
```

```
            r1, r2 = np.random.rand(2)
            A1 = 2*a*r1 - a
            C1 = 2*r2

            D_alpha = abs(C1*alpha[j] - wolves[i,j])
            X1 = alpha[j] - A1*D_alpha

            r1, r2 = np.random.rand(2)
            A2 = 2*a*r1 - a
            C2 = 2*r2

            D_beta = abs(C2*beta[j] - wolves[i,j])
            X2 = beta[j] - A2*D_beta

            r1, r2 = np.random.rand(2)
            A3 = 2*a*r1 - a
            C3 = 2*r2

            D_delta = abs(C3*delta[j] - wolves[i,j])
            X3 = delta[j] - A3*D_delta

            wolves[i,j] = (X1 + X2 + X3) / 3

        # Keep areas within bounds
        wolves[i] = np.clip(
            wolves[i], lower, upper
        )
    print(
        "Iteration:", t+1,
```

```
            "Best Fitness:", round(alpha_score, 4)
        )
    return alpha, alpha_score

# 5. Execute Optimization
best_areas, best_fitness = GWO()

weight, stresses, U = analyze_truss(best_areas)

print("\n----- OPTIMIZED TRUSS DESIGN -----")

for i, area in enumerate(best_areas):
    print(
        "Member", i+1,
        "Area =", round(area, 7), "m2"
    )
print("\nMinimum Weight:", round(weight, 4), "kg")
print("\nMember Stresses (MPa):")
for i, stress in enumerate(stresses):
    print(
        "Member", i+1,
        ":", round(stress/1e6, 3)
    )

print("\nNodal Displacements (m):")
print(U)
print("\nStress constraint satisfied:",
      np.all(np.abs(stresses) <= stress_limit))
print("Displacement constraint satisfied:",
```

np.max(np.abs(U)) <= disp_limit)

# ⇨OUTPUT

```
Iteration: 157 Best Fitness: 18.008
Iteration: 158 Best Fitness: 18.008
Iteration: 159 Best Fitness: 18.008
Iteration: 160 Best Fitness: 18.008
Iteration: 161 Best Fitness: 18.008
Iteration: 162 Best Fitness: 18.008
Iteration: 163 Best Fitness: 18.008
Iteration: 164 Best Fitness: 18.008
Iteration: 165 Best Fitness: 18.008
Iteration: 166 Best Fitness: 18.008
Iteration: 167 Best Fitness: 18.008
Iteration: 168 Best Fitness: 18.008
Iteration: 169 Best Fitness: 18.008
Iteration: 170 Best Fitness: 18.008
Iteration: 171 Best Fitness: 18.008
Iteration: 172 Best Fitness: 18.008
Iteration: 173 Best Fitness: 18.008
Iteration: 174 Best Fitness: 17.902
Iteration: 175 Best Fitness: 17.902
Iteration: 176 Best Fitness: 17.902
Iteration: 177 Best Fitness: 17.902
Iteration: 178 Best Fitness: 17.902
Iteration: 179 Best Fitness: 17.902
Iteration: 180 Best Fitness: 17.902
Iteration: 181 Best Fitness: 17.902
Iteration: 182 Best Fitness: 17.902
Iteration: 183 Best Fitness: 17.902
Iteration: 184 Best Fitness: 17.902
Iteration: 185 Best Fitness: 17.902
Iteration: 186 Best Fitness: 17.8945
Iteration: 187 Best Fitness: 17.8422
Iteration: 188 Best Fitness: 17.8422
Iteration: 189 Best Fitness: 17.8422
Iteration: 190 Best Fitness: 17.8422
Iteration: 191 Best Fitness: 17.8422
Iteration: 192 Best Fitness: 17.8422
Iteration: 193 Best Fitness: 17.8355
Iteration: 194 Best Fitness: 17.8218
Iteration: 195 Best Fitness: 17.8218
Iteration: 196 Best Fitness: 17.8218
Iteration: 197 Best Fitness: 17.8218
Iteration: 198 Best Fitness: 17.8218
Iteration: 199 Best Fitness: 17.8218
Iteration: 200 Best Fitness: 17.8218

----- OPTIMIZED TRUSS DESIGN -----
Member 1 Area = 0.0001336 m2
Member 2 Area = 0.0002402 m2
Member 3 Area = 0.0002403 m2

Minimum Weight: 17.7968 kg

Member Stresses (MPa):
Member 1 : 249.507
Member 2 : -250.151
Member 3 : -250.024

Nodal Displacements (m):
[ 0.          0.          0.00499014  0.          0.00249301 -0.00708194]

Stress constraint satisfied: False
Displacement constraint satisfied: True
```